UNIVERSITY OF LEEDS

COMP1850 Programming

Week 6 Session 1

Class Collaboration & Inheritance

Recap

UNIVERSITY OF LEEDS

Last time, we

- Considered how type hinting and type checking can be used to document classes better and make them more robust
- Looked at how we can write setter methods that validate attempts to modify the value of a field (though we cannot enforce use)
- Explored the use of dunder methods to customize the ways in which object behave in particular situations (e.g., printing, or use with operators such as +, `in`, [ ], etc)

Today's Aims

UNIVERSITY OF LEEDS

- Understand the distinction between equality of identity and equality of state, and see how we can test for the latter in classes
- Explore further the idea of classes that collaborate to solve a problem
- Investigate the concept of **inheritance**, and how we can build new classes from existing classes using this concept

Reminder: `Point` Class

UNIVERSITY OF LEEDS

```
class Point:
    """
    A point in 2D space.
    """

    def __init__(self, x, y):
        """
        Creates a Point, given x & y coordinates.
        """
        self.x = x
        self.y = y
    ...
```

What happens when we try to test `Point` objects for equality?

Task 1

UNIVERSITY OF LEEDS

`week1.6/session1/task1/`

1. cd to this directory in the terminal window
2. Run the Python interpreter by entering `python`
3. Enter these commands at the `>>>` prompt:

```
from point import Point
p = Point(1, 2)
q = Point(1, 2)
p == q
```

4. Press `Ctrl+D` to exit the interpreter

What's Going On?

UNIVERSITY OF LEEDS

- The objects referenced by variables p and q are in the same **state**, but they have different **identities**
- The default behaviour of == for instances of user-defined classes is to test for **equality of identity** – i.e., whether the two variables are referencing the same object
- Normally, what we want is for == to test for **equality of state** – i.e., whether the fields of the objects have the same values
- We achieve this by overriding dunder method `__eq__()`

Overriding __eq__()

UNIVERSITY OF LEEDS

```
class Point:
    ...
    def __eq__(self, other):
        """
        Tests whether this Point has the same coordinates as another.
        """
        if not isinstance(other, Point):
            return False

        return self.x == other.x and self.y == other.y
```

Note: we assume here that x and y coordinates have integer values

Task 2

UNIVERSITY OF LEEDS

`week1.6/session1/task2/`

1. cd to this directory in the terminal window
2. Run the Python interpreter by entering `python`
3. Enter these commands at the `>>>` prompt:

```
from point import Point
p = Point(1, 2)
q = Point(1, 2)
p == q
```

4. Press `Ctrl+D` to exit the interpreter

This new version of `Point` overrides `__eq__()`

`p == q` should now be `True`

Class Relationships

UNIVERSITY OF LEEDS

- OO software is composed of multiple classes
- Those classes can relate to each other in different ways
- One very strong type of relationship has one class acting as a parent to others, providing them with code they can use
- Another type of relationship has one class providing a component part that another can use
- Looser associations are also possible
- We will consider examples of all of these today...

Inheritance

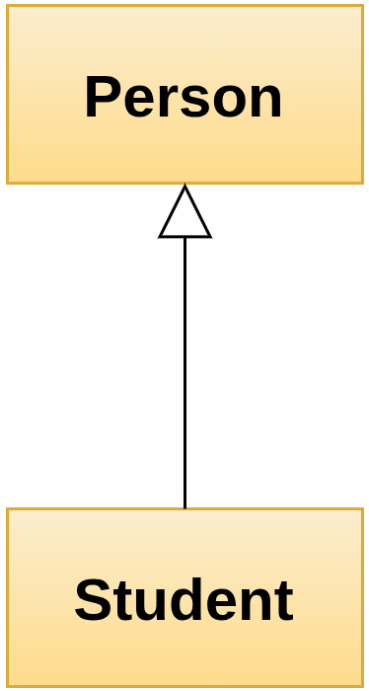
UNIVERSITY OF LEEDS

- In OO languages, we can create new classes by extending the features and functionality of existing classes
- The class we are extending is known as the **superclass** and the new class is known as the **subclass**
  - The terms **base class** & **derived class** are sometimes used
- A subclass **inherits** the attributes and behaviour of its superclass, and adds its own attributes and behaviour
- A subclass can also replace behaviour inherited from the superclass

‘Is a Kind Of’

UNIVERSITY OF LEEDS

- When a subclass inherits from a superclass, this models the relationship ‘is a kind of’ between the corresponding real-world classes
- Example: a Student is a kind of Person; students have the same attributes and behaviour as people, but they add new things:
  - New attributes such as university, degree programme, duration of degree, etc
  - New behaviour such as attend lecture, submit coursework, sit exam, etc



How Does This Work in Python?

UNIVERSITY OF LEEDS

- When defining a subclass, its name should be followed by parentheses containing the name of the superclass
- **Subclass constructor should invoke the superclass constructor**, to ensure that fields inherited from the superclass are created and initialized correctly:

```
super().__init__(arg1, arg2)
```

`super()` allows us to reference the superclass without naming it

In this example, the superclass constructor expects to be supplied with two arguments

Example

UNIVERSITY OF LEEDS

```
class Die:
    def __init__(self, sides):
        self.sides = sides
        self.value = 1

    def roll(self):
        self.value = random.randint(1, self.sides)
        return self.value


class D6(Die):
    def __init__(self):
        super().__init__(6)
```

See `week1.6/examples` for the full code

Task 3

UNIVERSITY OF LEEDS

`week1.6/session1/task3/`

- Study the provided `Point` & `Shape` classes
- Create a `Circle` class that inherits properly from `Shape`
- Write a demo program for `Circle`
- See `README.md` for full details of what is required

Composition & Association

UNIVERSITY OF LEEDS

- One class can act as a ‘building block’ for another without there being an inheritance relationship
- You’ve seen an example of this already: the `Circle` class uses the `Point` class to represent the centre coordinates of a circle, **but it doesn't inherit from the `Point` class**
- This is another form of software reuse, sometimes described as ‘reuse through composition’
- Note that we don’t always think of classes as components of other classes; sometimes the collaboration is looser than that

An Extended Example

UNIVERSITY OF LEEDS

The scenario:

- The United States Geological Survey operates an Earthquake Hazards Program in which it collates and publishes seismometer data to the web as various **data feeds**
- Feeds are available in different formats, including CSV
- Feeds cover different **time periods** (past hour, day, week, month)
- Feeds have different **significance levels** (e.g., ‘all’, ‘magnitude 4.5 or above’, ‘significant events only’)

An official website of the United States government Here's how you know



Significant Earthquakes, Past 30 days

Significant Earthquakes Archive

What makes an earthquake "significant"?

6.3 170 km NW of Port-Olry, Vanuatu
2026-10-24 02:35:57 (UTC)
Pager Alert Level: Green
MMI: IV (Light Shaking) 10.0 km

6.9 197 km WNW of Port-Olry, Vanuatu
2026-10-24 02:31:29 (UTC)
Pager Alert Level: Green
MMI: V (Moderate Shaking) 10.0 km

7.0 191 km ENE of Nagqu, China
2026-10-22 14:33:40 (UTC)
Pager Alert Level: Orange
MMI: VIII (Severe Shaking) 18.0 km

HOME
EARTHQUAKES
HAZARDS
SCIENCE
MONITORING

Earthquake Hazards Program

The USGS monitors and reports on earthquakes, assesses earthquake impacts and hazards, and conducts targeted research on the causes and effects of earthquakes. We undertake these activities as part of the larger National Earthquake Hazards Reduction Program (NEHRP), a four-agency partnership established by Congress.

Search Earthquake Catalog

Was this page helpful?

↩ Feeds and Notifications

Real-time Notifications

Earthquake Notification Service

Tweet Earthquake Dispatch

Real-time Feeds

ATOM

KML

Spreadsheet

QuakeML

GeoJSON Summary

GeoJSON Detail

For Developers

Spreadsheet Format

Description

CSV is a “comma separated values” text file. See the wikipedia for more information on this format.

This feed adheres to the USGS Earthquakes Feed Lifecycle Policy.

Usage

A simple text format suitable for loading data into spreadsheet applications like Microsoft Excel™. This is a good option for manual scientific analysis.

Output

Below are the fields included in the spreadsheet output:

- time
- latitude
- longitude
- depth

Feeds

Past Hour

Updated every minute.

- Significant Earthquakes
- M4.5+ Earthquakes
- M2.5+ Earthquakes
- M1.0+ Earthquakes
- All Earthquakes

Past Day

Updated every minute.

- Significant Earthquakes
- M4.5+ Earthquakes
- M2.5+ Earthquakes
- M1.0+ Earthquakes
- All Earthquakes

Our Goal

UNIVERSITY OF LEEDS

- We want some software that can download data from a user-specified earthquake data feed and analyze it in different ways
- We want to follow an object-oriented approach
- What classes do we need? What responsibilities will these classes have? How will they collaborate to achieve the goal?
- We will need multiple classes here; the Single Responsibility Principle applies in OOP as well as procedural programming...

A Possible Approach

UNIVERSITY OF LEEDS

`Quake`

- **Responsibilities**: represent relevant details of a single earthquake

`QuakeFeed`

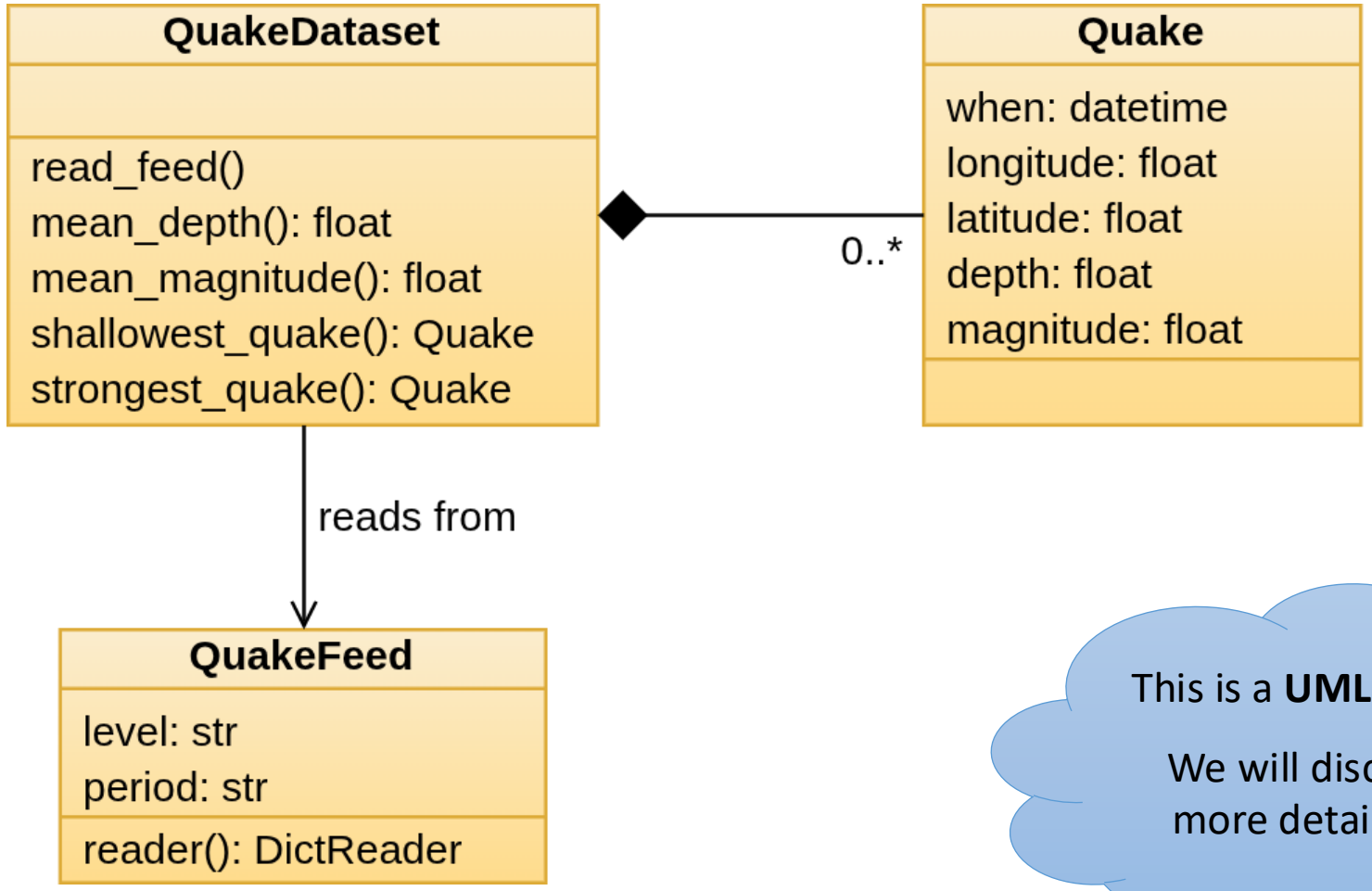
- **Responsibilities**: represent the chosen feed and provide access to its data

`QuakeDataset`

- **Responsibilities**: store downloaded data and facilitate analysis of it
- **Collaborations**: uses a `QuakeFeed` object to represent a feed and access its data; uses `Quake` objects to represent the details of individual earthquakes

Graphical Representation

UNIVERSITY OF LEEDS



This is a **UML class diagram**

We will discuss these in more detail next year...

Quake as a Data Class

UNIVERSITY OF LEEDS

When classes are needed primarily as a way of bundling related items of data together, Python, like many OO languages, provides more concise syntax for defining them:

```
from dataclasses import dataclass
from datetime import datetime

@dataclass
class Quake:
    when: datetime
    longitude: float
    latitude: float
    depth: float
    magnitude: float
```

Note use of `@dataclass` **decorator**, and how the fields are specified with names & types

A constructor is created for us!

QuakeFeed

UNIVERSITY OF LEEDS

- **Stop working on the tasks and pay attention!**
- We will walk you through the implementation of QuakeFeed
- If you have any questions, please ask...

Task 4

UNIVERSITY OF LEEDS

`week1.6/session1/task4/`

- Experiment with the `Quake` & `QuakeFeed` classes
- See `README.md` for full details of what to do

QuakeDataset

UNIVERSITY OF LEEDS

- **Stop working on the tasks and pay attention!**
- We will discuss some of the features of `QuakeDataset`
- If you have any questions, please ask...

Task 5

UNIVERSITY OF LEEDS

`week1.6/session1/task5/`

- Complete the implementation of `QuakeDataset`
- Write a program that uses the classes, in `quakeinfo.py`
- See `README.md` and comments in the files for full instructions

Reflection

UNIVERSITY OF LEEDS

- Could we design this differently? (Does `QuakeDataset` have too many reponsibilities? Could we have chosen different classes?)
- What would a solution look like if we adopted a procedural approach instead of an OO approach?
- What would a solution look like without user-defined classes or functions? How easy to understand would it be? What would we lose by implementing it in this way?

Summary

UNIVERSITY OF LEEDS

We have

- Distinguished between equality of identity and equality of state, and seen how to test for the latter in user-defined classes
- Discussed how we can create classes in Python that inherit from existing classes, adding new fields & methods or replacing methods
- Worked through a practical example of how an OO approach involves creating classes that collaborate to solve a problem

Learning More

UNIVERSITY OF LEEDS

How to Think Like a Computer Scientist

- Section 18.3: Sameness of Objects
- Chapter 19: Inheritance